

PESS-Project1

最终版 Write-up

张仕达

个人负责方向：Perf 性能分析与瓶颈定位

课程项目	PESS Project1 - bitarray.c 升级优化
作者	张仕达（学号：23010022094）
小组成员	程传哲、袁东霖、张仕达、郭晓伟
版本说明	测试版本 34 tiers；最终版本 45 tiers
完成日期	2026 年 4 月

一、实验背景与版本说明

本实验围绕 `bitarray.c` 中 `bitarray_rotate()` 的性能优化展开，要求在不改变测试框架和外部接口的前提下，对位数组子区间循环旋转进行正确性验证与性能提升。项目限制不允许使用多线程、SIMD、内联汇编，也不允许调用 `bitarray_new()` 创建新的 `bitarray` 作为大块辅助空间，因此优化必须在原位完成。

项目推进过程中形成了两个关键版本：测试版本可在大规模性能测试中稳定通过 tier 34；在此基础上，小组进一步把反转子过程从“逐 bit 交换”升级到“64 位分块交换 + 边界修正”，最终版本达到 tier 45。也就是说，本次升级的本质，是在已经将复杂度由 $O(n*k)$ 降为 $O(n)$ 之后，再继续削减常数因子，使 `reverse_subarray()` 成为真正高效的热点实现。

二、测试版本与最终版本对比

对比两个压缩包可见，`tests/mytests` 在两个版本中保持一致，说明最终版的性能跃迁不是依赖“删测试”或“放宽正确性要求”，而是来自 `bitarray.c` 的进一步工程化优化。

测试版本已经采用“三次翻转”思想：先将右旋统一换算成左旋，再通过翻转前半段、翻转后半段、翻转整个区间来完成旋转，从而把原始的 $O(n*k)$ 下降为 $O(n)$ 。最终版本则继续优化 `bitarray_reverse_subarray()`，让中间对齐部分直接以 `uint64_t` 为单位做成块翻转，仅在首尾不对齐区域保留逐 bit 交换。

此外，最终版还把 `bitarray_new()` 的缓冲区大小改为按 64 位上取整分配，为后续的整块访问提供更稳定的内存边界。这一改动虽然不改变接口语义，但明显体现出实现已经从“算法正确”推进到“数据布局与热点函数协同优化”的阶段。

版本	<code>./everybit -s</code>	<code>./everybit -m</code>	<code>./everybit -l</code>
测试版本（34 tiers）	21	29	34
最终版本（45 tiers）	30	40	45
提升幅度	+9	+11	+11

三、从 34 tiers 到 45 tiers 的改进策略

结合两个版本的实际代码差异，可以把本次升级总结为以下五条策略：

改进层面	从 34 tiers 升级到 45 tiers 的策略
算法层	保留“三次翻转实现旋转”的总体框架，但将翻转函数从逐 bit 交换升级为“边界逐 bit + 中

	间 64 位整块翻转”的混合策略，把热点从大量 <code>bitarray_get()/bitarray_set()</code> 调用转移为更少的整字操作。
数据布局层	<code>bitarray_new()</code> 的底层缓冲区不再只按字节上取整，而是按 64 位对齐分配，为 <code>uint64_t</code> 分块处理创造条件，减少跨字访问带来的额外开销。
实现层	新增 <code>reverse_bits_64()</code> ，将整块翻转时的位序反转压缩为固定几步位运算；同时使用 XOR 差分交换处理首尾零散 bit，避免额外临时数组。
验证层	保持 <code>tests/default</code> 与自定义 <code>tests/mytests</code> 全部通过，并重点回归极短区间、负数旋转、跨字节区间、非整字节长度和连续两次旋转等高风险场景。
结果层	实测从测试版 <code>./everybit -l</code> 只能通过 tier 34，提升到最终版通过 tier 45；说明优化不只是“常数级微调”，而是把关键热点路径进一步压缩后带来了可观的性能跃迁。

四、小组四人分工

本次 `bitarray.c` 升级由四人协作完成。为了保证四份最终版 `write-up` 都能体现个人贡献，同时避免内容完全割裂，团队采取“统一项目目标 + 各自重点展开”的写法。分工如下：

成员	学号	本次分工	在当前文档中的重点描述
程传哲	23040032008	负责代码集成与最终提交，完成 <code>bitarray.c</code> 的版本收敛、结果汇总，以及最终版 <code>write-up</code> 组织。	本成员负责内容在团队分工中交叉配合。
袁东霖	23020006170	提出并论证“三次翻转 + 分块翻转”的核心优化方案，完成旋转逻辑与反转逻辑的算法设	本成员负责内容在团队分工中交叉配合。

		计。	
张仕达	23010022094	使用 perf 与分档运行结果定位性能瓶颈，整理测试版与最终版的性能证据，明确后续优化应聚焦 reverse_subarray() 热点。	本人的 write-up 对该方向展开详细说明。
郭晓伟	23060021010	设计并维护 20 个自定义测试样例，覆盖边界情况与跨字节场景，负责最终版回归验证。	本成员负责内容在团队分工中交叉配合。

五、我的负责内容与最终版 write-up

我的个人负责方向：负责 perf 测试、原始性能瓶颈定位、版本间性能结果整理，以及“为什么还要在三次翻转基础上继续优化”的证据归纳。

个人负责内容

在本次升级中，我主要负责性能分析链路。首先对测试版本执行 ./everybit -s、./everybit -m、./everybit -l，确认其性能边界分别为 21、29、34 tiers；随后对最终版本重复相同流程，确认性能边界提升到 30、40、45 tiers。通过这一组结果可以直接证明：最终优化确实发生在性能关键路径，而不是偶然波动。

结合 bitarray.c 源码，我进一步确认测试版本虽然已经使用三次翻转，但 reverse_subarray() 仍逐 bit 调用 bitarray_get() 与 bitarray_set()。这说明当区间很长时，热点不再主要来自“旋转次数太多”，而是来自“翻转内部的单 bit 交换太密集”。最终版本正是顺着这一瓶颈继续下钻，把中间部分升级为 64 位分块翻转。

因此，我在小组中的作用不是直接改写全部代码，而是给出可信的性能证据，帮助团队判断：下一步优化不能停留在算法名义复杂度上，而要继续压缩 reverse_subarray() 的常数开销。

性能对比解读

从 tier 34 到 tier 45 的提升说明最终版对于大输入尤其有效。因为越到后面的 tier，位串长度和旋转规模越大，整块翻转能够节省的循环与位操作越多，所以收益会在高 tier 区间被放大。

这也解释了为什么最终版在 -s、-m、-l 三档测试上都提升明显：小数据下开销更低，大数据下热点更集中，两个方向同时受益。

六、性能分析与技术路线

原始代码的性能瓶颈集中在 `bitarray_rotate()` 路径上：右旋先被统一转为左旋，而左旋又会多次调用逐位移动逻辑。每一次移动都涉及大量 `bitarray_get()` 与 `bitarray_set()` 调用，造成了高频的位定位、读写和循环控制开销。

引入三次翻转之后，复杂度已经从与旋转量相关的 $O(n*k)$ 降为 $O(n)$ ；但在测试版本中，`reverse_subarray()` 仍然采用单 bit 交换，因此当区间很大时，热点依然集中在大量离散位操作上。最终版本针对这一点继续优化，把中间大段转换为 64 位整块处理，从而显著减少函数调用、条件判断与跨字节位搬运次数。

七、测试验证与结果说明

本组保留并使用自定义 `mytests` 作为回归测试集。该测试集共 20 个样例，覆盖长度为 1 和 2 的极小区间、旋转量为 0、等于长度、大于长度、负数旋转、子区间旋转、跨字节边界、非整字节长度以及连续两次旋转等情况。

两版代码均可通过 `tests/default` 与 `tests/mytests`；差异主要体现在性能测试中：测试版 `./everybit -s / -m / -l` 分别通过 21 / 29 / 34 tiers，而最终版提升到 30 / 40 / 45 tiers。

代表性测试样例

- t 1: n 1, r 0 1 7, e 1 —— 验证长度为 1 时旋转任意次数结果不变。
- t 7: n 10010110, r 0 8 9, e 01001011 —— 验证“旋转量大于区间长度”的模运算处理。
- t 10: n 11110000, r 2 4 -1, e 11100100 —— 验证中间子区间右旋/左旋换算是否正确。

- t 16: n 10000000000000001, r 0 16 -4, e 0000000000011000 —— 验证跨字节长区间旋转。
- t 19: 先对中间 7 位右旋 20，再对全串左旋 5 —— 验证连续两次旋转后的稳定性。

八、结论

综合来看，测试版到最终版的升级不是推翻重写，而是一次的“先做对、再做快”的迭代：先用三次翻转把旋转问题降到线性复杂度，再用 64 位分块反转进一步缩短热点路径。

这种升级策略的价值在于：既没有破坏接口与测试语义，也没有引入额外位数组作为辅助空间，却成功把大规模性能边界从 34 tiers 推进到 45 tiers，说明此次优化具有明确的工程效果和可复现性。

九、附录实验截图以及耗时表

耗时表

设计	编码	测试样例设计	测试调试	会议
1h	1h	30min	40min	35min

初始代码

```

void bitarray_rotate(bitarray_t* const bitarray,
                    const size_t bit_offset,
                    const size_t bit_length,
                    const ssize_t bit_right_amount) {
    assert(bit_offset + bit_length <= bitarray->bit_sz);

    if (bit_length == 0) {
        return;
    }

    // Convert a rotate left or right to a left rotate only, and eliminate
    // multiple full rotations.
    bitarray_rotate_left(bitarray, bit_offset, bit_length,
                        modulo(-bit_right_amount, bit_length));
}

static void bitarray_rotate_left(bitarray_t* const bitarray,
                                const size_t bit_offset,
                                const size_t bit_length,
                                const size_t bit_left_amount) {
    if (bit_length <= 1 || bit_left_amount == 0 || bit_left_amount == bit_length) {
        return;
    }

    bitarray_reverse_subarray(bitarray, bit_offset, bit_left_amount);
    bitarray_reverse_subarray(bitarray, bit_offset + bit_left_amount,
                            bit_length - bit_left_amount);
    bitarray_reverse_subarray(bitarray, bit_offset, bit_length);
}

```

```

static void bitarray_reverse_subarray(bitarray_t* const bitarray,
                                     const size_t bit_offset,
                                     const size_t bit_length) {
    if (bit_length <= 1) {
        return;
    }

    size_t left = bit_offset;
    size_t right = bit_offset + bit_length - 1;
    while (left < right) {
        const bool left_bit = bitarray_get(bitarray, left);
        const bool right_bit = bitarray_get(bitarray, right);
        bitarray_set(bitarray, left, right_bit);
        bitarray_set(bitarray, right, left_bit);
        left++;
        right--;
    }
}

```



```
ccz@chengchuanzhe: ~/projex X + v
inflating: everybit/bitarray.h
inflating: everybit/clint.py
inflating: everybit/ktiming.c
inflating: everybit/ktiming.h
inflating: everybit/main.c
inflating: everybit/Makefile
inflating: everybit/tests.c
inflating: everybit/tests.h
creating: everybit/tests/
inflating: everybit/tests/default
inflating: everybit/tests/mytests
inflating: everybit/bitarray.c.bak
extracting: everybit/.buildmode
inflating: everybit/bitarray.o
inflating: everybit/ktiming.o
inflating: everybit/main.o
inflating: everybit/tests.o
inflating: everybit/everybit
ccz@chengchuanzhe:~/projects$ ls
everybit
ccz@chengchuanzhe:~/projects$ find ~/projects -type d | grep everybit
/home/ccz/projects/everybit
/home/ccz/projects/everybit/tests
ccz@chengchuanzhe:~/projects$ cd everybit
ccz@chengchuanzhe:~/projects/everybit$ sed -i 's/\r$//' tests/default tests/mytests 2>/dev/null || true
ccz@chengchuanzhe:~/projects/everybit$ make clean
make: Warning: File 'Makefile' has modification time 19359 s in the future
rm -f everybit *.o .buildmode *.gcov *.gcno *.gcda
make: warning: Clock skew detected. Your build may be incomplete.
ccz@chengchuanzhe:~/projects/everybit$
```

```
ccz@chengchuanzhe: ~/projex X + v
ccz@chengchuanzhe:~/projects$ ls
everybit
ccz@chengchuanzhe:~/projects$ find ~/projects -type d | grep everybit
/home/ccz/projects/everybit
/home/ccz/projects/everybit/tests
ccz@chengchuanzhe:~/projects$ cd everybit
ccz@chengchuanzhe:~/projects/everybit$ sed -i 's/\r$//' tests/default tests/mytests 2>/dev/null || true
ccz@chengchuanzhe:~/projects/everybit$ make clean
make: Warning: File 'Makefile' has modification time 19359 s in the future
rm -f everybit *.o .buildmode *.gcov *.gcno *.gcda
make: warning: Clock skew detected. Your build may be incomplete.
ccz@chengchuanzhe:~/projects/everybit$ find . -type f -exec touch {} +
ccz@chengchuanzhe:~/projects/everybit$ make DEBUG=1 EXTRA_LDFLAGS='-fuse-ld=lld'
clang -std=c99 -Wall -m64 -g -O0 -o bitarray.o -c bitarray.c
bitarray.c:238:13: warning: unused function 'bitarray_rotate_left_one' [-Wunused-function]
238 | static void bitarray_rotate_left_one(bitarray_t* const bitarray,
    |
1 warning generated.
clang -std=c99 -Wall -m64 -g -O0 -o ktiming.o -c ktiming.c
clang -std=c99 -Wall -m64 -g -O0 -o main.o -c main.c
clang -std=c99 -Wall -m64 -g -O0 -o tests.o -c tests.c
tests.c:267:14: warning: variable length array folded to constant array as an extension [-Wgnu-folding-constant]
267 | const double fibs[FIB_SIZE] = {1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 676
    |
5, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, 514229, 832040, 1346269, 2178309, 3524578, 5702887, 922746
5, 14930352, 24157817, 39088169, 63245986, 102334155, 165580141, 267914296, 433494437, 701408733, 1134903170, 1836311903
, 2971215073, 4807526976, 7778742049, 12586269025, 20365011074, 32951280099, 53316291173, 86267571272};
    |
1 warning generated.
clang bitarray.o ktiming.o main.o tests.o -flto -fuse-ld=gold -lrt -fuse-ld=lld -o everybit
ccz@chengchuanzhe:~/projects/everybit$
```



```
ccz@chengchuanzhe: ~/projex  X + v
1 warning generated.
clang bitarray.o ktiming.o main.o tests.o -flto -fuse-ld=gold -lrt -fuse-ld=lld -o everybit
ccz@chengchuanzhe:~/projects/everybit$ ls -l everybit
-rwxr-xr-x 1 ccz ccz 31080 Mar 19 00:30 everybit
ccz@chengchuanzhe:~/projects/everybit$ ./everybit
./everybit -t tests/default
./everybit -t tests/mytests
usage: ./everybit
    -s Run a sample small (0.01s) rotation operation
    -m Run a sample medium (0.1s) rotation operation
    -l Run a sample large (1s) rotation operation
        (note: the provided -[s/m/l] options only test performance and NOT correctness.)
    -t tests/default      Run all tests in the testfile tests/default
    -n 1 -t tests/default Run test 1 in the testfile tests/default
Testing file tests/default.

Running test #0...
--> tests/default at line 34: PASS

Running test #1...
--> tests/default at line 41: PASS

Running test #2...
--> tests/default at line 49: PASS
--> tests/default at line 52: PASS
--> tests/default at line 55: PASS
--> tests/default at line 58: PASS
--> tests/default at line 61: PASS
Done testing file tests/default.
100101101001011010000101Testing file tests/mytests.
```

```
ccz@chengchuanzhe: ~/projex  X + v

Running test #0...
--> tests/mytests at line 26: PASS

Running test #1...
--> tests/mytests at line 31: PASS

Running test #2...
--> tests/mytests at line 36: PASS

Running test #3...
--> tests/mytests at line 41: PASS

Running test #4...
--> tests/mytests at line 46: PASS

Running test #5...
--> tests/mytests at line 51: PASS

Running test #6...
--> tests/mytests at line 56: PASS

Running test #7...
--> tests/mytests at line 61: PASS

Running test #8...
--> tests/mytests at line 66: PASS

Running test #9...
--> tests/mytests at line 71: PASS
```

[illegible]

```

ccr@chengchuanchu: ~/jreps × + v
Successfully completed tier: 24
----- END RESULTS -----
----- RESULTS -----
Tier 0 (#08) completed in 0.000001s
Tier 1 (#08) completed in 0.000001s
Tier 2 (#12) completed in 0.000001s
Tier 3 (#18) completed in 0.000001s
Tier 4 (#28) completed in 0.000001s
Tier 5 (#40) completed in 0.000002s
Tier 6 (#60) completed in 0.000002s
Tier 7 (#118) completed in 0.000003s
Tier 8 (#180) completed in 0.000005s
Tier 9 (#290) completed in 0.000007s
Tier 10 (#478) completed in 0.000012s
Tier 11 (#768) completed in 0.000018s
Tier 12 (#1230) completed in 0.000039s
Tier 13 (#1998) completed in 0.000047s
Tier 14 (#3238) completed in 0.000078s
Tier 15 (#5223) completed in 0.000140s
Tier 16 (#8458) completed in 0.000201s
Tier 17 (#140) completed in 0.000346s
Tier 18 (#240) completed in 0.000570s
Tier 19 (#300) completed in 0.000902s
Tier 20 (#500) completed in 0.001310s
Tier 21 (#900) completed in 0.001330s
Tier 22 (#1400) completed in 0.002451s
Tier 23 (#2300) completed in 0.003621s
Tier 24 (#3000) completed in 0.005020s
Tier 25 (#6200) completed in 0.009384s
Tier 26 (#10100) completed in 0.015001s
Tier 27 (#16000) completed in 0.023607s
Tier 28 (#26000) completed in 0.039021s
Tier 29 (#40000) completed in 0.064399s
Tier 30 (#60000) exceeded 0.18s cutoff with time 0.103475s
Successfully completed tier: 29
----- END RESULTS -----
----- RESULTS -----
Tier 0 (#08) completed in 0.000001s
Tier 1 (#08) completed in 0.000000s
Tier 2 (#12) completed in 0.000000s
Tier 3 (#18) completed in 0.000001s
Tier 4 (#28) completed in 0.000001s
Tier 5 (#40) completed in 0.000001s
Tier 6 (#60) completed in 0.000001s
Tier 7 (#118) completed in 0.000002s
Tier 8 (#180) completed in 0.000003s
Tier 9 (#290) completed in 0.000005s
Tier 10 (#478) completed in 0.000008s
Tier 11 (#768) completed in 0.000013s
Tier 12 (#1230) completed in 0.000020s
Tier 13 (#1998) completed in 0.000031s

```

```
ccz@chengchuanzhe:~/projects$ ./jsgps
Tier 27 (#168KB) completed in 0.023607s
Tier 28 (#265KB) completed in 0.039621s
Tier 29 (#438KB) completed in 0.064390s
Tier 30 (#696KB) exceeded 0.18s cutoff with time 0.183475s
Successfully completed tier: 29
----- END RESULTS -----
----- RESULTS -----
Tier 0 (#0B) completed in 0.000001s
Tier 1 (#8B) completed in 0.000000s
Tier 2 (#1B) completed in 0.000000s
Tier 3 (#1B) completed in 0.000001s
Tier 4 (#2B) completed in 0.000001s
Tier 5 (#4B) completed in 0.000001s
Tier 6 (#6B) completed in 0.000001s
Tier 7 (#11B) completed in 0.000002s
Tier 8 (#18B) completed in 0.000003s
Tier 9 (#28B) completed in 0.000005s
Tier 10 (#47B) completed in 0.000008s
Tier 11 (#76B) completed in 0.000013s
Tier 12 (#123B) completed in 0.000020s
Tier 13 (#198B) completed in 0.000031s
Tier 14 (#323B) completed in 0.000048s
Tier 15 (#522B) completed in 0.000077s
Tier 16 (#846B) completed in 0.000125s
Tier 17 (#140B) completed in 0.000209s
Tier 18 (#242B) completed in 0.000321s
Tier 19 (#396B) completed in 0.000515s
Tier 20 (#568B) completed in 0.000848s
Tier 21 (#90B) completed in 0.001092s
Tier 22 (#146B) completed in 0.002262s
Tier 23 (#236B) completed in 0.003701s
Tier 24 (#386B) completed in 0.005942s
Tier 25 (#624B) completed in 0.009480s
Tier 26 (#1016B) completed in 0.015414s
Tier 27 (#168KB) completed in 0.024688s
Tier 28 (#265KB) completed in 0.039701s
Tier 29 (#438KB) completed in 0.064312s
Tier 30 (#696KB) completed in 0.100751s
Tier 31 (#1MB) completed in 0.160631s
Tier 32 (#1MB) completed in 0.261862s
Tier 33 (#2MB) completed in 0.414982s
Tier 34 (#4MB) completed in 0.627875s
Tier 35 (#7MB) exceeded 1.08s cutoff with time 1.099655s
Successfully completed tier: 34
----- END RESULTS -----
ccz@chengchuanzhe:~/projects/everybits$
```

最终代码

```
static inline uint64_t reverse_bits_64(uint64_t x) {
    x = ((x & 0x5555555555555555ULL) << 1) | ((x >> 1) & 0x5555555555555555ULL);
    x = ((x & 0x3333333333333333ULL) << 2) | ((x >> 2) & 0x3333333333333333ULL);
    x = ((x & 0x0F0F0F0F0F0F0F0FULL) << 4) | ((x >> 4) & 0x0F0F0F0F0F0F0FULL);
    return __builtin_bswap64(x);
}

static void bitarray_reverse_subarray(bitarray_t* const bitarray,
                                     const size_t bit_offset,
                                     const size_t bit_length) {
    if (bit_length <= 1) return;

    size_t left = bit_offset;
    size_t right = bit_offset + bit_length - 1;
    uint64_t* buf = (uint64_t*)bitarray->buf;

    while (left < right && (left & 63) != 0) {
        size_t l_word = left >> 6;
        size_t r_word = right >> 6;
        uint64_t diff = ((buf[l_word] >> (left & 63)) ^ (buf[r_word] >> (right & 63)));
        buf[l_word] ^= (diff << (left & 63));
        buf[r_word] ^= (diff << (right & 63));
        left++;
        right--;
    }
}
```

```

if (left + 128 <= right) {
    size_t r_shift = (right - 63) & 63;
    uint64_t r_lower_mask = (1ULL << r_shift) - 1;
    uint64_t r_upper_mask = ~r_lower_mask;

    while (left + 128 <= right) {
        size_t l_word = left >> 6;
        size_t r_word = (right - 63) >> 6;

        uint64_t l_val = buf[l_word];

        uint64_t r_val = (r_shift == 0) ? buf[r_word] :
            (buf[r_word] >> r_shift) | (buf[r_word + 1] << (64 - r_shift));

        uint64_t rev_l = reverse_bits_64(l_val);
        uint64_t rev_r = reverse_bits_64(r_val);

        buf[l_word] = rev_r;

        if (r_shift == 0) {
            buf[r_word] = rev_l;
        } else {
            buf[r_word] = (buf[r_word] & r_lower_mask) | (rev_l << r_shift);
            buf[r_word + 1] = (buf[r_word + 1] & r_upper_mask) | (rev_l >> (64 - r_shift));
        }

        left += 64;
    }

    left += 64;
    right -= 64;
}

while (left < right) {
    size_t l_word = left >> 6;
    size_t r_word = right >> 6;
    uint64_t diff = ((buf[l_word] >> (left & 63)) ^ (buf[r_word] >> (right & 63)));
    buf[l_word] ^= (diff << (left & 63));
    buf[r_word] ^= (diff << (right & 63));
    left++;
    right--;
}
}

```



```
_mask
1] =
Tier 32 (~1MB) completed in 0.002650s
Tier 33 (~2MB) completed in 0.004119s
Tier 34 (~4MB) completed in 0.006566s
Tier 35 (~7MB) completed in 0.010887s
Tier 36 (~12MB) completed in 0.017072s
Tier 37 (~19MB) completed in 0.027794s
Tier 38 (~31MB) completed in 0.045411s
Tier 39 (~51MB) completed in 0.070971s
Tier 40 (~83MB) completed in 0.118383s
Tier 41 (~135MB) completed in 0.195562s
Tier 42 (~218MB) completed in 0.302060s
Tier 43 (~354MB) completed in 0.490051s
Tier 44 (~573MB) completed in 0.806327s
Tier 45 (~927MB) exceeded 1.00s cutoff with time 1.321670s
Successfully completed tier: 44
---- END RESULTS ----
[ perf record: Woken up 10 times to write data ]
[ perf record: Captured and wrote 2.526 MB perf.data (66175 samples) ]
ccz@chengchuanzhe:~/projects/everybit$
1) return;
```